14.10 — Параметризованные SQL-запросы и SqlParameter

Конспект

Параметризованный запрос содержит обозначения вроде @id и @price. Их значения передают через Parameters отдельно от SQL-текста. Пользовательский ввод не нужно вставлять в SQL конкатенацией.

SqlParameter хранит имя, SQL-тип и значение параметра. Parameters.AddWithValue() добавляет параметр и определяет тип по значению C#. Parameters.Add() позволяет явно задать SqlDbType и размер строки.

Параметры защищают передачу значений от SQL-инъекции и позволяют работать со строками, датами и числами без ручного добавления кавычек. Параметром передают значение, а не имя таблицы или SQL-оператор.

Соответствия типов: int - INT, decimal - DECIMAL, string - NVARCHAR, bool - BIT. Для SQL NULL используется DBNull.Value. Размер и точность параметра должны соответствовать столбцу базы.

Ответы на вопросы для самопроверки

1. Что такое параметризованный SQL-запрос?
Запрос, в котором значения представлены параметрами и передаются отдельно от SQL-текста.

2. Для чего используется SqlParameter?
Для задания имени, типа и значения параметра команды.

3. Почему нельзя вставлять значения прямо в SQL-строку?
Можно получить SQL-инъекцию, ошибки кавычек и ошибки формата чисел или дат.

4. Что такое SQL-инъекция?
Попадание пользовательского ввода в SQL-код так, что он меняет смысл запроса и выполняет нежелательные действия.

5. Чем WHERE id = 5 отличается от WHERE id = @id?
В первом варианте значение записано в SQL, во втором его передают отдельно через параметр @id.

6. Как передать значение параметра в SqlCommand?
Добавить параметр в command.Parameters и задать его Value, например command.Parameters.Add("@id", SqlDbType.Int).Value = 5;

7. Можно ли использовать параметры в INSERT?
Да, например VALUES (@full_name, @phone, @email).

8. Можно ли использовать параметры в UPDATE?
Да, например SET price = @price WHERE id = @id.

9. Можно ли использовать параметры в DELETE?
Да, например WHERE id = @id.

10. Для чего используется AddWithValue()?
Для добавления параметра с именем и значением; тип определяется по переданному значению.

11. Зачем явно указывать SqlDbType?
Чтобы передавать значение в нужном SQL-типе и избежать нежелательного определения типа или преобразования.

12. Как передать SQL NULL?
Установить Value = DBNull.Value. Обычный C# null не является SQL NULL.

Практическая часть

Ниже для каждого запроса указаны параметры, типы и пример значений из C#. Переменная connection обозначает уже открытый SqlConnection. Длины строк и точность DECIMAL в исходном задании не заданы, поэтому для примера используются NVARCHAR(150), NVARCHAR(30) и DECIMAL(10,2); при другой структуре базы их нужно согласовать со столбцами.

1. Поиск товара

@id: SqlDbType.Int, значение C# int productId = 5. Метод выполнения - ExecuteReader().

```
int productId = 5;
string sql = "SELECT id, name, price FROM products WHERE id = @id;";
using SqlCommand command = new SqlCommand(sql, connection);
command.Parameters.Add("@id", SqlDbType.Int).Value = productId;
using SqlDataReader reader = command.ExecuteReader();
```

2. Изменение цены

@id: SqlDbType.Int, значение 3 из int productId. @price: SqlDbType.Decimal, значение 220.00m из decimal price. Метод - ExecuteNonQuery().

```
int productId = 3;
decimal price = 220.00m;
string sql = "UPDATE products SET price = @price WHERE id = @id;";
using SqlCommand command = new SqlCommand(sql, connection);
command.Parameters.Add("@id", SqlDbType.Int).Value = productId;
SqlParameter priceParameter =
    command.Parameters.Add("@price", SqlDbType.Decimal);
priceParameter.Precision = 10;
priceParameter.Scale = 2;
priceParameter.Value = price;
int changedRows = command.ExecuteNonQuery();
```

3. Добавление клиента

@full_name: SqlDbType.NVarChar, string fullName = "Иван Петров". @phone: SqlDbType.NVarChar, string phone = "+79990000000". @email: SqlDbType.NVarChar, string? email = null, поэтому передаётся DBNull.Value. Метод - ExecuteNonQuery().

```
string fullName = "Иван Петров";
string phone = "+79990000000";
string? email = null;
string sql = @"INSERT INTO customers (full_name, phone, email)
VALUES (@full_name, @phone, @email);";
using SqlCommand command = new SqlCommand(sql, connection);
command.Parameters.Add("@full_name", SqlDbType.NVarChar, 150)
    .Value = fullName;
command.Parameters.Add("@phone", SqlDbType.NVarChar, 30)
    .Value = phone;
command.Parameters.Add("@email", SqlDbType.NVarChar, 150)
    .Value = (object?)email ?? DBNull.Value;
int addedRows = command.ExecuteNonQuery();
```

4. Удаление клиента

@id: SqlDbType.Int, значение C# int customerId = 5. Метод - ExecuteNonQuery(). Если на клиента ссылаются заказы, удаление может быть запрещено внешним ключом.

```
int customerId = 5;
string sql = "DELETE FROM customers WHERE id = @id;";
using SqlCommand command = new SqlCommand(sql, connection);
command.Parameters.Add("@id", SqlDbType.Int).Value = customerId;
int deletedRows = command.ExecuteNonQuery();
```

Каждый фрагмент рассматривается отдельно. Для типов используются пространства имён System.Data и Microsoft.Data.SqlClient.